

Benchmark de Paralelismo com Multiprocessing em Python

Análise fiscal distribuída: comparando execução sequencial vs. paralela em
16 milhões de notas fiscais

Programação Concorrente e Distribuída — ADSN04 · Lucas Vasconcelos
Pessoa de Oliveira & João Gabriel Lucas Pinheiro de Lima · Prof. Rafael ·
23/06/2026

0 Problema: Processar 16 Milhões de Notas Fiscais

Base sintética em CSV com **16.000.000 de registros (~1,9 GB)**. Objetivo: comparar execução sequencial vs. paralela.

Precisão Monetária

Conversão com `Decimal` e parse real de datas com `datetime.strptime`

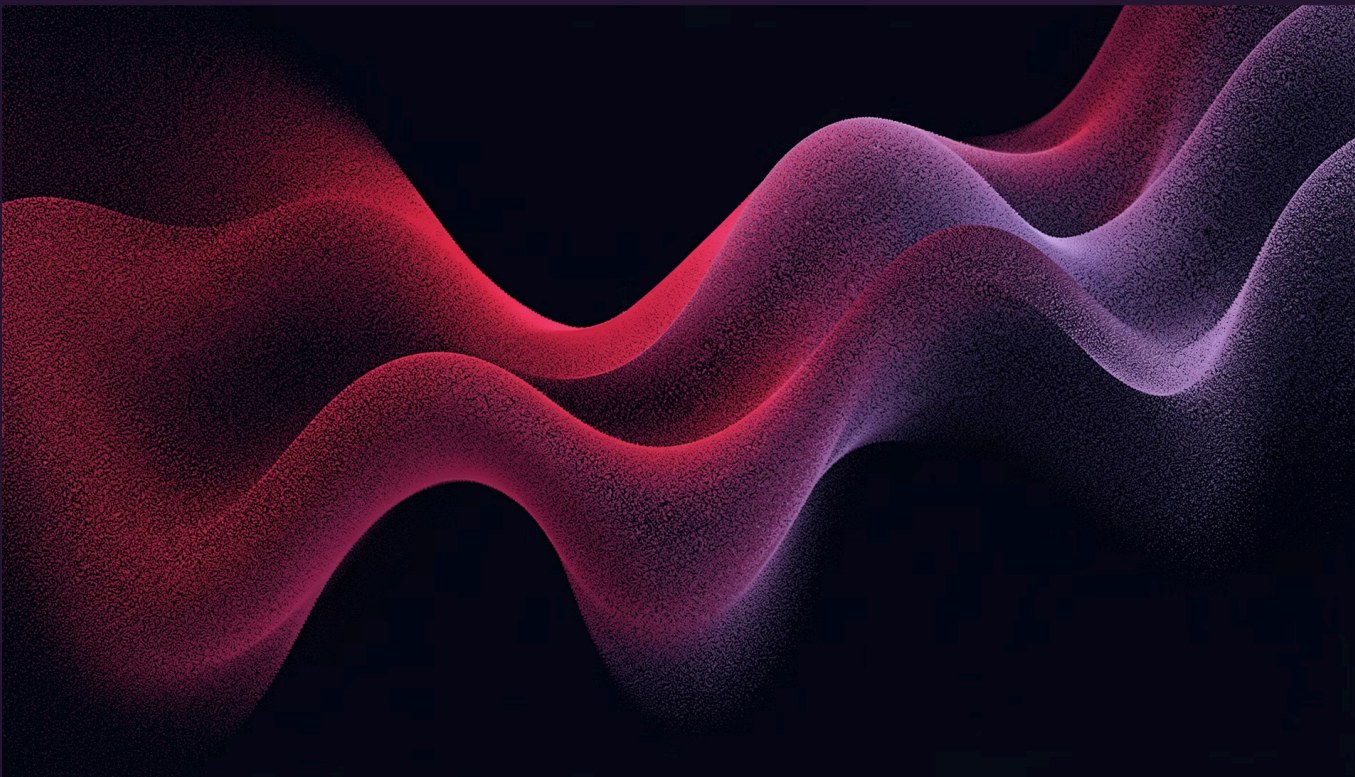
Cálculo Fiscal

ICMS por alíquota, validação cruzada de `valor_total` e classificação de risco fiscal

Agregações Complexas

Por produto, estado, cidade, CNPJ, categoria e mês — rankings com `heapq`

A Base de Dados: notas_fiscais.csv



Escala do Dataset

Registros	16.000.000
Tamanho	~1,9 GB
Colunas	13 campos
Período	2022–2024
CNPJs emitentes	5.000 únicos
Produtos sintéticos	~4.000 combinações
Estados / Cidades	10 / 50
Itens por NF	1 a 6 (aleatório)

Campos: item_id, nf_id, data, estado, cidade, cnpj_emitente, categoria, produto, quantidade, preco_unitario, desconto, aliquota_icms, valor_total

Ambiente Experimental

Hardware e stack utilizados para garantir reprodutibilidade e comparabilidade dos testes.

Hardware

Processador	AMD Ryzen 7 5700X — 3,40 GHz
Núcleos	8 físicos / 16 threads (SMT)
RAM	32 GB
Armazenamento	932 GB
SO	Windows 11 — 64 bits

Software e Metodologia

Python 3.12
CPython com multiprocessing nativo
Medição Precisa
time.perf_counter() em todas as configurações
Volume Constante
Mesmos 16 milhões de registros em cada teste

Estratégia de Paralelização

MAP-REDUCE COM MULTIPROCESSING.POOL



Divisão por bytes

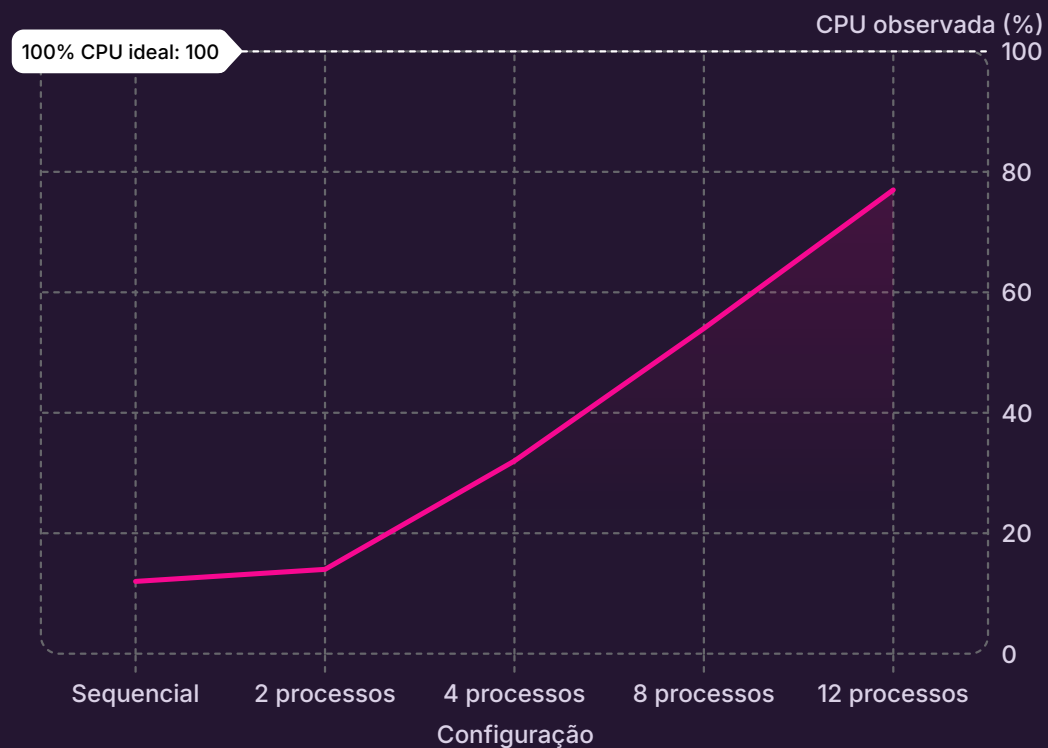
Map

Cálculo local

Reduce

O trabalho por linha é completamente independente — sincronização ocorre apenas na etapa final de **Reduce**, minimizando contenção entre processos.

Evidências de Execução: Uso Real de CPU



Sequencial → ~12%

Um único núcleo ativo

CPU cresce com paralelismo

O uso de CPU escala de forma consistente com o número de processos, processando os mesmos **16 milhões de registros** em cada teste.

12 processos → ~77%

Quase 8 núcleos físicos saturados

Resultados: Tempo, Speedup e Eficiência

Processos	Tempo (s)	Speedup	Eficiência
Sequencial	237,7843	1,00×	—
2	133,1444	1,79×	89,5%
4	68,9927	3,45×	86,2%
8	38,5264	6,17×	77,1%
12 (melhor)	34,5666	6,88×	57,3%

34,57s

Melhor tempo

Com 12 processos

6,88×

Speedup máximo

Sobre 16M registros

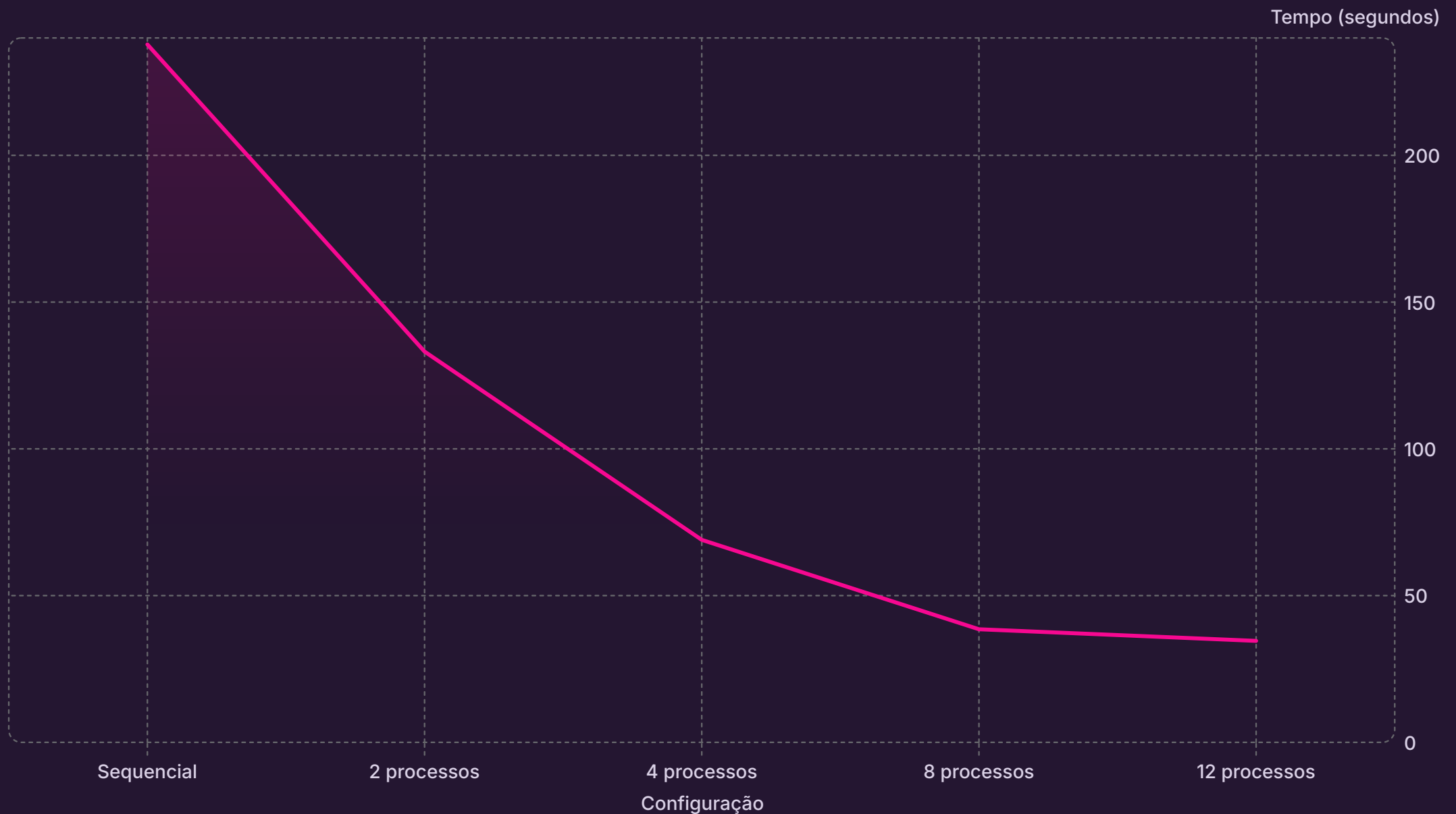
85,5%

Redução no tempo

De 237,78s para 34,57s

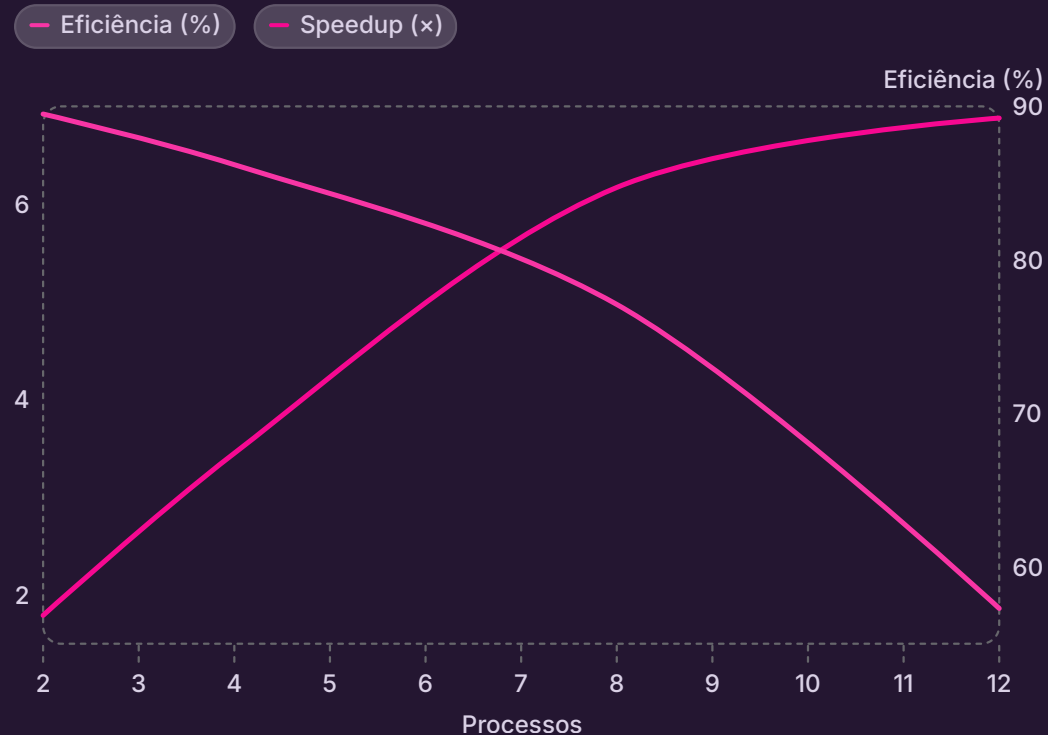
$\text{Speedup}(p) = T_{\text{sequencial}} / T(p)$ | $\text{Eficiência}(p) = \text{Speedup}(p) / p \times 100\%$

Tempo de Execução por Configuração



Queda de **71%** entre Sequencial e 4 processos. A partir de 8 processos, o ganho marginal diminui — sinal claro de saturação e overhead crescente.

Speedup e Eficiência: A Lei de Amdahl em Ação



Padrão esperado e confirmado

O speedup cresce de forma **sublinear** — sempre abaixo do ideal. A eficiência cai suavemente, sem degraus abruptos.

Speedup sublinear

Sempre abaixo de p (speedup ideal = nº de processos)

Eficiência decrescente

Queda suave desde o processo 6 — $R^2 = 0,986$ no modelo de Amdahl

Análise e Conclusão

Spawn no Windows

Sem fork nativo, custo de criação de processos é significativo

Pickle + Decimal

Serialização IPC agravada pela precisão monetária

Contenção de I/O

Leitura concorrente de 1,9 GB / 16M linhas

Parsing CPU-bound

Conversões por linha limitam o ganho paralelo

85,5%

Mais rápido

Redução no tempo total

6,88×

Speedup final

Com 12 processos

0,986

R² do modelo

Amdahl + overhead